



Universidade Federal de Sergipe
Departamento de Computação

RELATÓRIO TÉCNICO

Atividade 1 (AV1) — Processos, Threads, Escalonamento e Inferência Local com Ollama

Disciplina: Sistemas Operacionais — 2026.2

Trilha A — Chat local: Ollama + Open WebUI

Equipe: Llama

Integrantes: Carlos Henrico Fontes Cabral, Daniel de Andrade Lima, Lauren Victoria Ferreira Roma, Marcos Vinícios Assunção Nery Santos, Roseane Resende Couto, Wanessa Silva Santos

Repositório GitHub:

https://github.com/LaurenVictoriaFR/SO_UFS_2026_2_Roma_Lauren_Santos_Marcos

Vídeo da atividade:

https://drive.google.com/file/d/1_Bv_BxVhiFWYNxPONXjoYaqvnVrElprZ/view?usp=drivesdk

São Cristóvão, Sergipe, setembro de 2026

Sumário

Sumário	2
1. Tabela indicativa do que cada componente da equipe fez na atividade	3
2. Introdução e problema	3
2.1 Pergunta norteadora	4
2.2 Objetivos específicos	4
3. Trilha e camada de aplicação selecionadas	4
3.1 Justificativa da escolha	4
3.2 Foco de observação da trilha	4
4. Modelo Hugging Face selecionado	5
4.1 Riscos e limitações conhecidos do modelo	6
5. URL do projeto público criado no GitHub	6
6. Ambiente experimental	6
6.1 Inventário do ambiente	6
7. Arquitetura da aplicação	9
7.1 Decisões de implantação	10
8. Processos, threads e chamadas de sistema	11
8.1 Processos e threads	11
8.2 Chamadas de sistema	14
9. Configurações e metodologia	15
9.1 Configuração 1 — Execução padrão	15
9.2 Configuração 2 — Concorrência ou carga controlada	16
9.3 Configuração 3 — Ajuste de execução local	16
9.4 Repetições e carga de teste	16
10. Resultados	17
10.1 Tempos e vazão	17
10.2 Uso de recursos e erros	18
10.3 Gráfico	19
11. Discussão	19
12. Relação com Sistemas Operacionais e IA generativa	20
13. Limitações e ameaças à validade	22
14. Declaração de uso de IA generativa	23
14.1 Prompts relevantes	23
14.2 Verificação e distribuição das contribuições	23
14.3 Distribuição das contribuições dos integrantes no uso de IA	24
15. Conclusão	24
16. Referências	25

1. Tabela indicativa do que cada componente da equipe fez na atividade

Integrante	Principais atividades realizadas
Carlos Henrico Fontes Cabral	Seção 9 (Configurações e metodologia) e Seção 10 (Resultados)
Daniel de Andrade Lima	Seção 4 (Modelo Hugging Face selecionado), Seção 7 (Arquitetura da aplicação) e Seção 8.1 (Processos e threads)
Lauren Victoria Ferreira Roma	Seção 3 (Trilha e camada de aplicação selecionadas), Seção 6 (Ambiente experimental) e Seção 8.2 (Chamadas de sistema); Responsável pela execução de todos os comandos técnicos e coleta de evidências experimentais utilizadas
Marcos Vinícios Assunção Nery Santos	Seção 2 (Introdução e problema) e Seção 15 (Conclusão)
Roseane Resende Couto	Seção 11 (Discussão)
Wanessa Silva Santos	Seção 12 (Relação com Sistemas Operacionais e IA generativa) e Seção 13 (Limitações e ameaças à validade)

2. Introdução e problema

Aplicações de Inteligência Artificial generativa executadas localmente transcendem a simples execução de um modelo de linguagem estático. Em um ambiente real, essas aplicações mobilizam e exigem o gerenciamento contínuo de processos, threads, memória RAM e de vídeo, armazenamento, comunicação em rede, além de interagir com arquivos, bibliotecas, runtimes e mecanismos de proteção do sistema operacional subjacente.

Neste contexto, o presente trabalho documenta a instalação, configuração e observação de um sistema de inferência local operando em um ambiente Linux via WSL. Para a realização dos experimentos, a equipe selecionou a Trilha A, utilizando a plataforma Open WebUI como camada de aplicação. O runtime obrigatório escolhido para carregar e atender às requisições locais foi o Ollama, operando de forma integrada ao contêiner da interface gráfica.

O modelo selecionado para os testes de inferência foi o meta-llama/Llama-3.2-3B-Instruct, atendendo estritamente à regra estrutural que exige a utilização de um modelo de linguagem com até 10 bilhões de parâmetros.

O escopo analítico deste relatório visa relacionar os conceitos teóricos de concorrência, escalonamento, filas de requisições e uso de recursos computacionais à prática de execução de uma aplicação moderna. O problema central que guia todas as observações, métricas e análises deste estudo é definido pela seguinte pergunta norteadora: como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução afetam processos, threads, uso de CPU, memória, armazenamento e responsividade de um sistema local de IA generativa?

A partir deste questionamento, o documento detalha o mapeamento de processos, a análise de chamadas de sistema (via strace) e os resultados de experimentos comparativos variando a carga de requisições e as configurações de execução, avaliando criticamente o comportamento do sistema sob diferentes condições de estresse.

2.1 Pergunta norteadora

Como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução afetam processos, threads, uso de CPU, memória, armazenamento e responsividade de um sistema local de IA generativa?

2.2 Objetivos específicos

- Mapear a árvore de processos (PID e PPID) e identificar as principais chamadas de sistema geradas pela comunicação entre o Open WebUI e o runtime do Ollama.
- Mensurar e comparar o consumo de memória RAM e de CPU durante as fases de inicialização, carregamento do modelo e inferência (geração de texto).
- Analisar o comportamento do escalonador do sistema operacional e a variação da latência (responsividade) quando a aplicação é submetida a diferentes cenários de concorrência e cargas de requisição controladas.
- Verificar o impacto no uso de recursos computacionais e no tempo total de requisição ao alterar parâmetros de execução local, como o tamanho do contexto de entrada enviado ao modelo.

3. Trilha e camada de aplicação selecionadas

Trilha escolhida: Trilha A — Chat local: Ollama + Open WebUI.

3.1 Justificativa da escolha

O modelo Llama3.2-3B oferece um bom equilíbrio de qualidade de respostas e requisito de hardware, facilitando a observação de chamadas de sistema e uso de recursos de hardware sem esgotar o desempenho computacional local. A escolha de quantização Q4 oferece um bom compromisso entre tamanho do modelo, velocidade de processamento e qualidade da resposta gerada, apropriado às limitações de hardware do ambiente usado.

A equipe optou pela Trilha A por ela operar localmente com requisições via HTTP, sem demandar de componentes extras de indexação ou embeddings, presentes nas Trilhas B e C. Como o foco da atividade está nos mecanismos de processos, threads e chamadas de sistema, evitar a complexidade extra de um pipeline de RAG possibilitou uma maior dedicação à análise desses mecanismos. Além disso, a Trilha A dispõe de uma maior facilidade de configuração e documentação mais ampla, o que reforçou essa escolha.

3.2 Foco de observação da trilha

- Processos do Ollama e do Open WebUI: documentados na Seção 8.1, ollama (PID 160), o filho llama-server (PID 1143), e o processo real do Open WebUI (python3 uvicorn, PID 427, visto via docker top).
- Comunicação HTTP pela API local: descrita na Seção 6.2, onde se observa o fluxo: Open WebUI → API do Ollama via HTTP → Ollama rodando no WSL2.
- Portas, sockets e conexões: as portas utilizadas são 11434 (Ollama) e 3000→8080 (Open WebUI). Na Seção 8.2, o strace mostrou as chamadas accept4 e poll, que correspondem à gestão desses sockets.
- Execução nativa ou em contêiner: o Ollama roda nativo no WSL2, enquanto o Open WebUI roda em contêiner Docker.

- Volumes, diretórios persistentes e logs: detalhados nas Seções 6.2 e 7.1.
- Consumo de memória durante carregamento e uso do modelo: VRAM entre ~2317-2327 MiB, RAM do processo Ollama ~21MB, tempo de carregamento de 78,76s em GPU contra 124-185s em CPU.
- Comportamento com múltiplas conversas/requisições simultâneas: a Configuração 2 mostrou que 3 requisições simultâneas não são processadas em paralelo, ficam em fila FIFO.

4. Modelo Hugging Face selecionado

- **Identificação:**
 - O modelo utilizado foi publicado pela organização Unsloth, utilizando como base o modelo original criado e treinado pela Meta. O nome completo é Llama-3.2-3B-Instruct-GGUF e a URL oficial do *model card* para garantir a rastreabilidade e reprodutibilidade do nosso ambiente é: <https://huggingface.co/unsloth/Llama-3.2-3B-Instruct-GGUF>.
- **Parâmetros:**
 - O modelo base possui 3 bilhões (3B) de parâmetros. Essa escolha atende perfeitamente à restrição da atividade, que exige um modelo com limite de até 10 bilhões de parâmetros, além de garantir um custo computacional reduzido para os testes.
- **Finalidade:**
 - Trata-se de um modelo focado em instrução (*Instruct*). É adequado para nossa camada de aplicação pois é otimizado para casos de uso conversacionais, diálogos multilíngue e recuperação de informações (*agentic retrieval*).
- **Idioma:**
 - O modelo oferece suporte oficial ao português, inglês, alemão, francês, italiano, hindi, espanhol e tailandês. Garantindo que a qualidade das entradas e saídas que vamos avaliar nos experimentos não sofra degradação por barreiras linguísticas.
- **Licença:**
 - Opera sob a *Llama 3.2 Community License*, que permite o uso responsável, mantendo nosso experimento em conformidade legal.
- **Formato e Quantização:**
 - O formato escolhido foi o GGUF com a quantização Q4_K_M (4 bits). Essa combinação foi escolhida por ser altamente compatível com o runtime Ollama e por oferecer o melhor compromisso entre economia de memória RAM, velocidade de processamento e qualidade da resposta gerada.
- **Contexto:**
 - O modelo possui uma ampla janela de contexto de até 128.000 tokens. Permitindo que os testes incluam ampla variação de cargas na memória com tamanhos de contexto diferentes.
- **Tamanho dos arquivos:**
 - O arquivo do modelo ocupa exatamente 2.02 GB, o que reduz o impacto de E/S no armazenamento e otimiza o tempo de carregamento na memória.
- **Requisitos de hardware:**
 - O tamanho de 2.02 GB é ideal para a viabilidade no nosso ambiente local, alinhando-se à recomendação de hardware de CPU e 8 GB de RAM para modelos de 1B a 3B com quantização leve.
- **Compatibilidade:**

- O formato GGUF é completamente compatível com Ollama, minimizando problemas técnicos relativos à execução local do modelo escolhido. Além disso, a página do modelo também oferece opções nativas de execução do modelo através do Ollama.
- **Documentação:**
 - Modelo possui documentação clara e extensiva, tanto na página de modelo do Unsloth quanto na página original da Meta. Além das informações básicas do modelo, a documentação existente também fornece informações como:
 - Arquitetura do modelo;
 - Casos de uso pretendidos e fora do escopo;
 - Tutorial de como utilizar o modelo com várias plataformas;
 - Informações de treinamento e
 - Impacto ambiental.

4.1 Riscos e limitações conhecidos do modelo

Como limitação inerente a LLMs locais desse porte, há o risco de alucinação e viés de dados em *prompts* mais complexos, exigindo interpretação crítica dos resultados por parte da equipe durante as inferências.

Do ponto de vista técnico, o Llama-3.2-3B-Instruct é um modelo estritamente texto-para-texto, sendo incapaz de visão computacional e pensamento presentes nas variantes maiores de 11B e 90B.

Ademais, somos limitados também pela licença comercial do modelo escolhido (*Llama 3.2 Community License Agreement*).

5. URL do projeto público criado no GitHub

Nome do repositório	SO_UFS_2026_2_Roma_Lauren_Santos_Marcos
URL do repositório	https://github.com/LaurenVictoriaFR/SO_UFS_2026_2_Roma_Lauren_Santos_Marcos
Commit / tag / release utilizado na entrega	88d3685
Licença do repositório	MIT License

6. Ambiente experimental

6.1 Inventário do ambiente

Item	Registro
Distribuição e versão do sistema operacional	Ubuntu 26.04.1 LTS (Resolute Raccoon)
Versão do kernel	6.18.33.2-microsoft-standard-WSL2
Processador e número de núcleos/threads	AMD Ryzen 5 6600H, 6 núcleos físicos / 12 threads
Memória RAM disponível	8GB host → 3,4 GiB visíveis ao WSL2

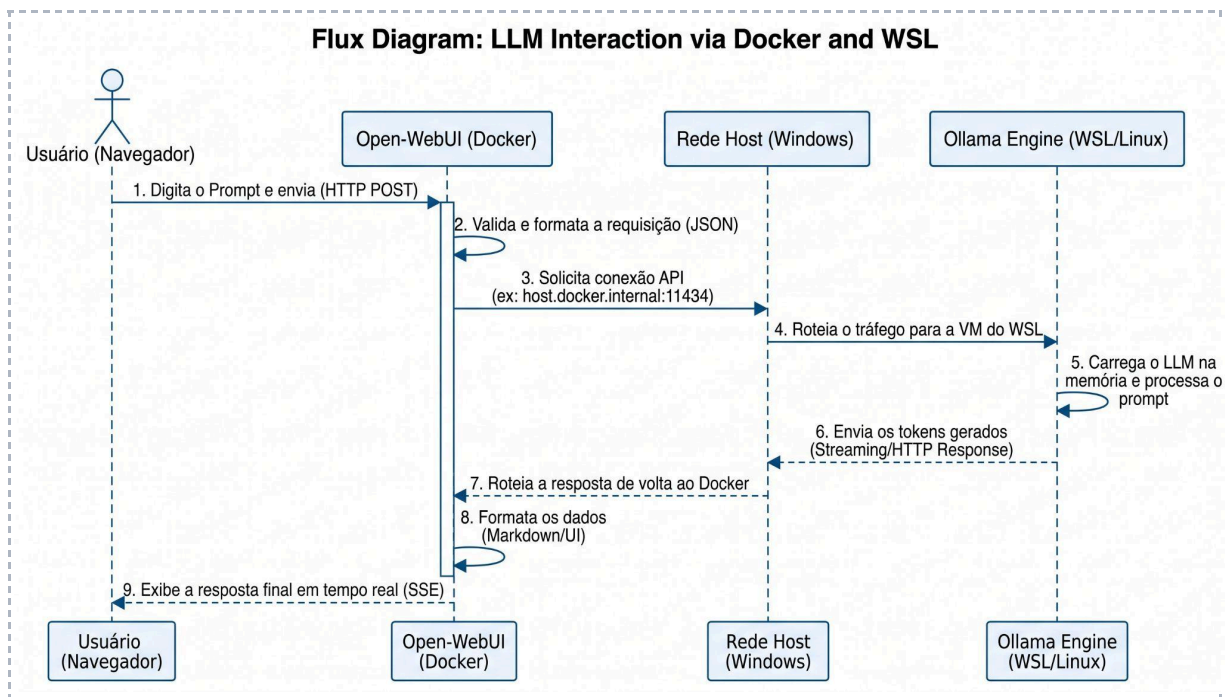
Item	Registro
GPU e VRAM, se houver	NVIDIA GeForce RTX 3050, 4096 MiB (4GB) VRAM, passthrough confirmado via nvidia-smi
Armazenamento disponível	950GB reportado pelo WSL2, mas o teto real é 22GB livres no C:\ do host (96% ocupado)
Versão do Ollama	0.33.2
Versão da linguagem e dependências da camada de aplicação	Python 3.11.16
Ambiente de execução: nativo, máquina virtual, WSL ou contêiner	WSL2

6.2 Ficha da camada de aplicação

Item	Registro
Nome e URL do repositório GitHub	Open WebUI — https://github.com/open-webui/open-webui
Autor ou organização	open-webui (org no GitHub)
Licença	- Open WebUI License - Base BSD-3-Clause - Clausula 4: Proíbe remover ou alterar a marca, mas, não se aplica a deployment de até 50 usuários em uma janela de 30 dias - Conclusão: A equipe são 6 pessoas usando localmente pra fins acadêmicos, isso cai dentro da exceção dos 50 usuários/30 dias, então estamos em conformidade sem precisar de permissão escrita nem licença paga.
Commit, tag ou release utilizado	2a960a59fe1dbbd35282f0556b3666d81102e781
Data de acesso	2026-09-04 (docker inspect)
Finalidade da aplicação	Interface de IA amigável ao usuário, compatível com Ollama, API OpenAI, entre outros.
Arquitetura simplificada	- Usuário → navegador → Open WebUI (container Docker, porta 3000, mapeada pra 8080 interna) - Open WebUI → API do Ollama via HTTP - Ollama roda nativo no WSL2, porta 11434, usa a RTX 3050 via passthrough - Duas redes diferentes envolvidas no caminho: a rede bridge do container Docker e a rede da distro WSL2 - Resposta volta pelo mesmo caminho até o navegador
Dependências	Python 3.11.16
Comandos de instalação e execução	Docker pull ghcr.io/open-webui/open-webui:main + docker run com bind na porta 3000→8080 e volume open-webui:/app/backend/data
Adaptações realizadas	O endereço sugerido pela documentação oficial (http://host.docker.internal:11434) não funcionou. Precisou

Item	Registro
	ser configurado manualmente o IP da interface do WSL2 (172.26.19.219:11434) na tela de conexões do Open WebUI
Limitações e erros encontrados	- Erro observado ao tentar host.docker.internal: Connection refused (via curl -v de dentro do container) - ss -tlnp confirmou que o Ollama escuta em todas as interfaces (*:11434) - O curl -v mostrou que host.docker.internal resolve corretamente para 192.168.65.254, mas essa não é a mesma sub-rede da distro WSL2 (172.26.19.219, confirmado via hostname -I) - Conclusão: Docker Desktop e a distro WSL2 operam em namespaces de rede distintos, mesmo com a integração WSL2 habilitada — a variável host.docker.internal não alcança o Ollama nesse cenário

7. Arquitetura da aplicação



O diagrama ilustra a arquitetura de comunicação e o fluxo de dados entre uma interface web e um modelo de linguagem (LLM) rodando localmente via Ollama. Cada etapa é definida como segue:

1. O usuário submete uma mensagem via navegador. Isso gera uma requisição HTTP POST padrão direcionada ao servidor do Open-WebUI.
2. O Open-WebUI processa essa entrada internamente. Ele valida os dados e os empacota em um formato JSON estruturado, que é o padrão esperado pela API do Ollama.
3. Como o Open-WebUI está em uma rede isolada do Docker, ele não pode acessar o "localhost" do Windows diretamente. Para resolver isso, ele faz uma chamada para host.docker.internal:11434
4. O sistema operacional hospedeiro (Windows) recebe essa requisição na porta 11434 e atua como um roteador. Ele identifica que o serviço escutando nessa porta está dentro da rede virtual do WSL (Linux) e encaminha os pacotes para a máquina virtual.
5. O Ollama Engine no WSL recebe o JSON, aloca recursos do sistema (trazendo os pesos do modelo LLM do armazenamento para a memória RAM ou VRAM) e inicia o processamento matemático do prompt.
6. À medida que o serviço gera cada token, ele imediatamente inicia o envio desses fragmentos de volta via *Streaming* através de uma resposta HTTP contínua.
7. A rede host do Windows faz o caminho inverso de volta, pegando o stream de dados saindo da VM do WSL e roteando-o de volta para a interface de rede do contêiner Docker.
8. O Open-WebUI recebe esse fluxo contínuo de tokens e os processa, aplicando formatações de interface (como renderização de tabelas e código em Markdown).
9. O *frontend* utiliza **SSE** (Server-Sent Events) para manter uma conexão unidirecional aberta com o navegador do usuário, permitindo que a resposta apareça na tela palavra por palavra de forma fluida, sem a necessidade de recarregar a página.

7.1 Decisões de implantação

Onde o Ollama é executado	Ollama executado em ambiente WSL.
Onde o Open WebUI é executado	A camada de aplicação do Open-WebUI roda em contêiner docker através da integração do Docker com WSL.
Porta do Ollama	A porta padrão 11434 foi utilizada.
Porta do Open WebUI	Porta 3000:8080 padrão do open-webui/docker.
Endereço do provedor Ollama configurado no Open WebUI	O provedor se encontra no endereço local 172.26.19.219:11434 (devido à limitações citadas em 6.2).
Volumes / diretórios persistentes	Open-WebUI: - Diretório de dados do contêiner: /app/backend/data - Dados brutos mapeados no WSL: /var/lib/docker/volumes/open-webui/_data Ollama: - Diretório de modelos: /usr/share/ollama/.ollama/models
Variáveis de ambiente relevantes	Open-WebUI: - WEBUI_SECRET_KEY: Validação e persistência de sessão; - OLLAMA_BASE_URL: Aponta o URL que a aplicação irá utilizar para se comunicar com o serviço do Ollama. Ollama: - OLLAMA_DEBUG: Ativa logs detalhados do servidor; - OLLAMA_HOST: Define a interface e a porta que o serviço utiliza; - OLLAMA_NUM_PARALLEL: Determina a quantidade máxima de processos em paralelo que um modelo pode processar; - OLLAMA_KEEP_ALIVE: Define a quantidade de tempo máxima permitida para que um modelo continue carregado na memória.

A arquitetura escolhida separa os serviços para otimizar o desempenho e a compatibilidade do sistema. Executar o Ollama diretamente no WSL garante que o serviço tenha acesso eficiente aos recursos de hardware da máquina (como a placa de vídeo) enquanto num ambiente linux, fundamentais para rodar os modelos de inteligência artificial. Paralelamente, isolar o Open WebUI em um contêiner Docker facilita a manutenção e evita conflitos. Para garantir a comunicação entre esses ambientes, utilizou-se o IP da rede virtual do WSL em vez do `localhost`, permitindo que o frontend no Docker conseguisse acessar o serviço do Ollama no host.

A persistência de dados foi estruturada através do mapeamento de volumes, para evitar a perda de informações a cada reinicialização do Docker. Ao mapear o diretório de dados do Open WebUI, o sistema garante a retenção do histórico de conversas e das configurações dos usuários. Da mesma forma, o mapeamento da pasta do Ollama é crítico para a usabilidade, pois impede que o sistema precise baixar novamente os pesados arquivos dos modelos de linguagem sempre que for iniciado.

Por fim, o uso das variáveis de ambiente reflete um foco em segurança e gerenciamento inteligente de hardware. Enquanto o frontend utiliza uma chave secreta para proteger as sessões dos usuários (embora essa chave seja opcional), o backend do Ollama emprega variáveis específicas (`KEEP\_ALIVE` e `NUM\_PARALLEL`) para otimizar o uso da memória RAM e VRAM. Essas configurações evitam o esgotamento dos recursos da máquina ao descarregar modelos inativos e limitar a quantidade de processamento simultâneo, mantendo a aplicação estável.

8. Processos, threads e chamadas de sistema

8.1 Processos e threads

- Comando utilizado:

```
ps -eo pid,ppid,stat,ni,pri,psr,pcpu,pmem,nlwp,comm --sort=-pcpu | grep -iE "ollama|llama|docker|containerd|open-webui|python"
```
- Processo real do Open WebUI:

```
python3 -m uvicorn open_webui.main:app, PID 427, PPID 404
```

visto via docker top open-webui (não aparece no ps do host porque roda em namespace de PID isolado do container)

PID	PPID	Estado	Nº threads	%CPU	%MEM	Comando
148	1	Ssl	18	0.4	0.6	ollama
1073	1072	Ssl+	14	0.3	0.8	docker-desktop-
427	404	-	-	-	-	python3 uvicorn open_webui
1135	148	Sl	24	92.5	26.2	llama-server

Esta tabela representa um *snapshot* do gerenciador de processos do sistema operacional, ilustrando a alocação de recursos e a hierarquia de execução durante a operação de uma arquitetura local de Inteligência Artificial baseada em LLM. Onde se evidencia a delegação de tarefas no sistema operacional: processos líderes de sessão operando em segundo plano (estado Ssl), como o daemon ollama (PID 148) e o gerenciador de contêineres docker-desktop (PID 1073), mantêm-se majoritariamente ociosos e em prontidão (estado *Interruptible Sleep*), consumindo menos de 1% de CPU enquanto escutam requisições originadas da interface open-webui.

O impacto computacional da pilha acaba inteiramente sobre o processo subordinado llama-server (PID 1135), que possui como processo pai o ollama (PID 148). Sob a lente de sistemas operacionais, o estado Sl (*Multi-threaded Sleep*) e a alocação de 24 *threads* (NLWP) para este processo indicam paralelização de cálculos matemáticos para inferência do modelo de linguagem. O consumo de 92,5% da CPU e 26,2% da memória RAM confirma que o modelo está ativamente computando e buscando parâmetros na memória.

Comando utilizado:

```
ps -eLf | grep -iE "ollama|llama-server"
```

UID	PID	PPID	LWP	C	NLWP
ollama	148	1	148	0	18
ollama	148	1	256	0	18
ollama	148	1	257	0	18
ollama	148	1	258	0	18
ollama	148	1	259	0	18

UID	PID	PPID	LWP	C	NLWP
ollama	148	1	260	0	18
ollama	148	1	261	0	18
ollama	148	1	262	0	18
ollama	148	1	263	0	18
ollama	148	1	264	0	18
ollama	148	1	274	0	18
ollama	148	1	273	0	18
ollama	148	1	294	0	18
ollama	148	1	870	0	18
ollama	148	1	871	0	18
ollama	148	1	872	0	18
ollama	148	1	931	0	18
ollama	148	1	932	0	18
ollama	1135	148	1135	94	24
ollama	1135	148	1137	0	24
ollama	1135	148	1138	0	24
ollama	1135	148	1141	0	24
ollama	1135	148	1143	0	24
ollama	1135	148	1144	0	24
ollama	1135	148	1145	0	24
ollama	1135	148	1146	0	24
ollama	1135	148	1147	0	24
ollama	1135	148	1148	0	24
ollama	1135	148	1149	0	24
ollama	1135	148	1150	0	24

(Nota: mostrando 12 das 24 threads do llama-server (comando original continha "head -30" resultando na visão limitada do comando — NLWP=24 confirma o total real))

Detalhamento da árvore de processos que demonstra o mapeamento das tarefas ao nível de Processos Leves (LWP) pelo escalonador do sistema. O daemon principal ollama (PID 148) sustenta 18 threads concorrentes em estado de prontidão e sem consumo imediato de processamento, atuando primariamente no gerenciamento assíncrono do serviço para escutar as requisições. A execução de fato acontece no processo-filho llama-server (PID 1135). Observa-se como o processo llama-server (PID 1135) instancia 24 threads (NLWP = 24) para lidar com a alta demanda computacional de inferência, com sua thread principal monopolizando até 94% dos ciclos do processador para coordenar as demais.

Comando utilizado:

```
pstree -p | grep -B2 -A20 -iE "ollama"
```

```

|                                     ^-{init-systemd(Ub)}(8)
|-networkd-dispatcher(159)
|-ollama(160)-+-llama-server(3149)-+-{llama-server}(3151)
|                                     |-{llama-server}(3152)
|                                     |-{llama-server}(3153)
|                                     |-{llama-server}(3155)
|                                     |-{llama-server}(3156)
|                                     |-{llama-server}(3157)
|                                     |-{llama-server}(3158)
|                                     |-{llama-server}(3159)
|                                     |-{llama-server}(3160)
|                                     |-{llama-server}(3161)
|                                     |-{llama-server}(3162)
|                                     |-{llama-server}(3163)
|                                     |-{llama-server}(3164)
|                                     |-{llama-server}(3165)
|                                     |-{llama-server}(3166)
|                                     |-{llama-server}(3167)
|                                     |-{llama-server}(3168)
|                                     |-{llama-server}(3172)
|                                     |-{llama-server}(3173)
|                                     |-{llama-server}(3174)
|                                     |-{llama-server}(3175)
|                                     |-{llama-server}(3176)
|                                     ^-{llama-server}(3177)
|                                     |-{ollama}(222)
|                                     |-{ollama}(224)
|                                     |-{ollama}(227)
|                                     |-{ollama}(228)
|                                     |-{ollama}(230)
|                                     |-{ollama}(239)
|                                     |-{ollama}(240)
|                                     |-{ollama}(246)
|                                     |-{ollama}(247)
|                                     |-{ollama}(248)
|                                     |-{ollama}(249)
|                                     |-{ollama}(250)
|                                     |-{ollama}(251)
|                                     |-{ollama}(252)
|                                     |-{ollama}(1170)
|                                     |-{ollama}(1171)
|                                     ^-{ollama}(1172)
|-polkitd(1475)-+-{polkitd}(1477)
|               |-{polkitd}(1478)
|               ^-{polkitd}(1479)
|-rsyslogd(181)-+-{rsyslogd}(204)
|               |-{rsyslogd}(205)
|               ^-{rsyslogd}(208)
|-systemd(899)---(sd-pam)(901)
|-systemd(394)---(sd-pam)(397)
|-systemd-journal(46)
|-systemd-logind(164)
|-systemd-resolve(71)
|-systemd-udevd(93)
^--unattended-upgr(209)---{unattended-upgr}(294)

```

(Hierarquia de processos do sistema)

A árvore de processos representa graficamente a estrutura de processos e *threads* gerenciada pelo sistema operacional, ilustrando a topologia de execução e a hierarquia de dependências a partir do processo de inicialização raiz. Um detalhe semântico central desta visualização é a utilização da notação de chaves {} para agrupar e denotar as threads (LWPs). Esta demarcação visual demonstra como as dezenas de instâncias subordinadas ao processo llama-server são ramificações de concorrência criadas para compartilhar o mesmo espaço de endereçamento de memória e recursos do processo-pai.

Comando utilizado:

```
top -H -b -n 1 | head -30
```

```
top - 23:36:02 up 27 min,  2 users,  load average: 2.14, 0.66, 0.45
Threads: 95 total,  7 running, 88 sleeping,  0 stopped,  0 zombie
%Cpu(s): 50.4 us, 15.9 sy,  0.0 ni, 30.3 id,  0.4 wa,  0.0 hi,  3.0 si,  0.0 st
MiB Mem : 3528.6 total,  48.8 free, 1567.1 used, 2134.2 buff/cache
MiB Swap: 1024.0 total,  661.4 free,  362.6 used. 1961.5 avail Mem

   PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
  1300 ollama    20   0   51.6g   1.0g 788036 R   61.1   30.4   0:37.24 llama-s+
  1322 ollama    20   0   51.6g   1.0g 788036 R   61.1   30.4   0:08.36 llama-s+
  1325 ollama    20   0   51.6g   1.0g 788036 R   61.1   30.4   0:08.34 llama-s+
  1326 ollama    20   0   51.6g   1.0g 788036 R   61.1   30.4   0:08.32 llama-s+
  1323 ollama    20   0   51.6g   1.0g 788036 R   55.6   30.4   0:08.34 llama-s+
  1324 ollama    20   0   51.6g   1.0g 788036 R   55.6   30.4   0:08.34 llama-s+
  1319 ollama    20   0   51.6g   1.0g 788036 S    5.6   30.4   0:00.08 llama-s+
  1333 lauren_+  20   0    8156   5568  3552 R    5.6    0.2   0:00.14 top
      1 root      20   0   24332   7292   5012 S    0.0    0.2   0:03.35 systemd
```

(Estado dos recursos de *hardware* no momento de execução)

Aqui é possível ver múltiplas threads da aplicação llama-server encontram-se ativamente no estado *Running*, cada qual demandando independentemente entre 55,6% e 61,1% de processamento, comprovando a distribuição da carga de inferência através de múltiplos núcleos físicos da CPU. Observa-se que todas as threads em execução reportam uma pegada estritamente idêntica: 51.6g de Memória Virtual (VIRT), 1.0g de Memória Residente (RES) e 788036 de Memória Compartilhada (SHR). Essa simetria demonstra que tais instâncias não são processos alocados de forma isolada, mas sim Processos Leves (LWPs) operando sob um único espaço de endereçamento compartilhado, o que elimina a necessidade de duplicação de dados. Além disso, a disparidade entre a memória virtual provisionada (51.6 GB) e a parcela efetivamente instanciada na RAM física (1.0 GB) revela o uso de mapeamento de arquivos em memória (*memory-mapped files*) para acesso otimizado aos tensores da quantização GGUF.

8.2 Chamadas de sistema

Chamada / família	Ocorrências (strace -c)	Categoria	Relevância observada
futex	5.407 chamadas (76,54% do tempo total, 510 erros)	Sincronização entre processos/threads	O futex consumiu 76,54% do tempo total registrado pelo strace durante a sessão monitorada. Como sua categoria é de sincronização entre threads, isso mostra que boa parte do tempo de processamento das

Chamada / família	Ocorrências (strace -c)	Categoria	Relevância observada
			requisições é gasto em coordenação e espera entre as threads que o llama-server gerencia, não em cálculo puro
clone / clone3	30 chamadas (28 + 2)	Criação de processos/threads	O Ollama usa o clone3 e o clone para criar o llama-server e suas 24 threads.
accept4 / poll	170 + 1.947 chamadas	Rede e comunicação local	O accept4 tem como papel aceitar conexões de rede chegando, e o poll fica na espera, checando se há dados prontos para ler/escrever. Eles são necessários para o servidor atender múltiplas requisições sem travar
mmap	453 chamadas	Alocação/mapeamento de memória	Ele mapeia o arquivo do modelo diretamente no espaço de memória, permitindo que o Ollama acesse os parâmetros sob demanda, sem precisar copiar todo o conteúdo do arquivo para a RAM de uma vez

As quatro chamadas de sistema selecionadas: futex, clone/clone3, accept4/poll e mmap, cobrem, em conjunto, quatro das sete categorias designadas: sincronização entre threads, criação de processos/threads, rede e comunicação local, e alocação/mapeamento de memória. Não foram escolhidas chamadas relacionadas à leitura/gravação de arquivos, criação de logs ou acesso a diretórios, já que o enunciado não impõe interpretar todas as famílias, apenas justificar as escolhidas.

O futex, encarregado de 76,54% do tempo total registrado, mostra que a maior parte do tempo de resposta é gasta em sincronização e espera entre as 24 threads do llama-server, não em processamento efetivo. As chamadas clone e clone3 marcam o momento em que o Ollama cria o processo llama-server e suas threads. Já accept4 e poll mostram como o servidor gerencia conexões de rede vindas do Open WebUI sem travar diante de múltiplas requisições. Por fim, o mmap explica como o Ollama carrega os parâmetros do modelo de forma eficaz, mapeando o arquivo na memória sob demanda, sem copiar tudo de uma vez.

Em conjunto, essas quatro chamadas mostram que a execução do sistema depende tanto de coordenação interna entre threads quanto de comunicação externa via rede, com o carregamento do modelo viabilizado por mecanismos de memória mapeada.

9. Configurações e metodologia

Os experimentos utilizam o mesmo modelo e a mesma camada de aplicação (Open WebUI) em todas as configurações, variando apenas os parâmetros descritos abaixo.

9.1 Configuração 1 — Execução padrão

Descrição: Execução padrão do sistema em aceleração por GPU (NVIDIA RTX 3050), mantendo as configurações nativas do Ollama e Open WebUI.

Carga de Teste: Executaram-se 3 repetições de prompt curto (~15-20 tokens de entrada) e 3 repetições de prompt longo (~80-90 tokens de entrada), além de uma execução inicial de referência sob carga fria (cold start).

Parâmetros e Métricas: Acompanharam-se o tempo de carregamento frio (cold load), a latência total HTTP de ponta a ponta (*tempo_total_http_s*), o tempo de processamento interno do Ollama (*tempo_total_ollama_ms*), a velocidade de geração em tokens por segundo (*tokens_por_seg*), o consumo de CPU/RAM visíveis e a alocação de VRAM.

9.2 Configuração 2 — Concorrência ou carga controlada

Parâmetro alterado	Número de requisições simultâneas submetidas à aplicação (1 requisição isolada vs. 3 requisições simultâneas).
Hipótese de impacto	Dada a limitação de hardware (4 GB VRAM e 3,4 GB RAM visíveis no WSL2), o Ollama não executará as requisições em paralelo real no hardware. Espera-se que o runtime enfileire as requisições sequencialmente, aumentando o tempo total percebido pelo usuário final proporcionalmente à sua posição na fila de espera.
Método / comandos utilizados	Abertura simultânea de 3 abas/conversas no Open WebUI e envio imediato do mesmo prompt longo nas três instâncias, monitorando no log do Ollama se o processamento é sobreposto ou estritamente sequencial.

9.3 Configuração 3 — Ajuste de execução local

Condição comparada	Execução em GPU dedicada (padrão) versus execução forçada em CPU (<i>CUDA_VISIBLE_DEVICES=-1</i>).
Hipótese de impacto	A inferência em CPU apresentará uma queda severa na taxa de geração de tokens (~2 a 5 tokens/s em CPU vs. ~35 a 40 tokens/s em GPU) e um aumento expressivo no tempo de carregamento inicial do modelo na memória RAM host.
Método / comandos utilizados	Parada do serviço <i>systemd</i> do Ollama (<i>sudo systemctl stop ollama</i>), inicialização manual da instância com a variável de ambiente <i>CUDA_VISIBLE_DEVICES=-1 ollama serve</i> , e execução dos 3 prompts curtos (mesmas versões da Configuração 1). Optou-se pelo prompt curto em vez do longo nesta configuração devido ao tempo inviável de execução em CPU (15x a 20x mais lento).

9.4 Repetições e carga de teste

Configuração	Tamanho de entrada/carga	Condição de concorrência	Nº de repetições
Config 1: Padrão (GPU)	Curto (~17 tokens) e Longo (~84 tokens)	Sequencial (1 por vez)	6 (3 curtos + 3 longos) + 1 Carga Fria
Config 2: Concorrência	Longo (~84 tokens)	Simultânea (3 requisições)	3 requisições em paralelo

Configuração	Tamanho de entrada/carga	Condição de concorrência	Nº de repetições
Config 3: CPU Forçada	Curto (~17 tokens)	Sequencial (1 por vez)	3 (com carga fria registrada)

10. Resultados

10.1 Tempos e vazão

Nº	Configuração	Horário	Prompt Tokens	Eval Tokens	Tempo Processamento (ms)	Tempo Total Http(s)	Tokens Por Seg
1	Config 1 (Padrão Curto)	18:10:00	22	45	2503,60	12,26	36,15
2	Config 1 (Padrão Curto)	18:12:00	17	55	3678,51	4,33	20,18
3	Config 1 (Padrão Curto)	18:15:00	18	45	2266,21	2,38	24,10
4	Config 1 (Padrão Longo)	18:15:00	102	889	25234,72	25,62	36,71
5	Config 1 (Padrão Longo)	18:17:00	97	716	19636,24	20,10	36,81
6	Config 1 (Padrão Longo)	18:18:00	99	886	26529,42	27,00	33,86
0	Config 1 (Padrão Carga Fria)	18:32:00	36	36	2235,97	81000	28,20
7	Config 2 (Concorrente 1)	18:51:34	117	934	26976,27	116	37,93
8	Config 2 (Concorrente 2)	18:51:37	1(cache)	795	20462,48	137	38,96
9	Config 2 (Concorrente 3)	18:51:35	1(cache)	465	11870,78	151	39,20

10	Config 3 (CPU-curto 1)	20:13:00	37	47	32138,69	158	2,04
11	Config 3 (CPU-curto 2)	20:19:00	17(20 cache)	57	21866,28	22,83	4,98
12	Config 3 (CPU-curto 3)	20:21:00	17(20 cache)	70	22787,87	23,40	5,27

10.2 Uso de recursos e erros

Durante o monitoramento dos processos na fase de inicialização e inferência, observou-se a presença do *container* Docker do Open WebUI atuando como cliente HTTP e do serviço da API do Ollama em segundo plano no sistema operacional. Para a execução das operações matriciais, o motor de inferência utiliza instâncias do mecanismo *runner* (llama.cpp). Quando alocado em aceleração por GPU, o sistema manteve cerca de 23 a 25 threads dedicadas à gestão da API HTTP, sincronização via CUDA e inter-process communication. Por outro lado, ao forçar a execução em CPU, o mecanismo inicializou exatamente 6 threads de processamento via OpenMP, número correspondente aos núcleos físicos do processador AMD Ryzen 5 6600H.

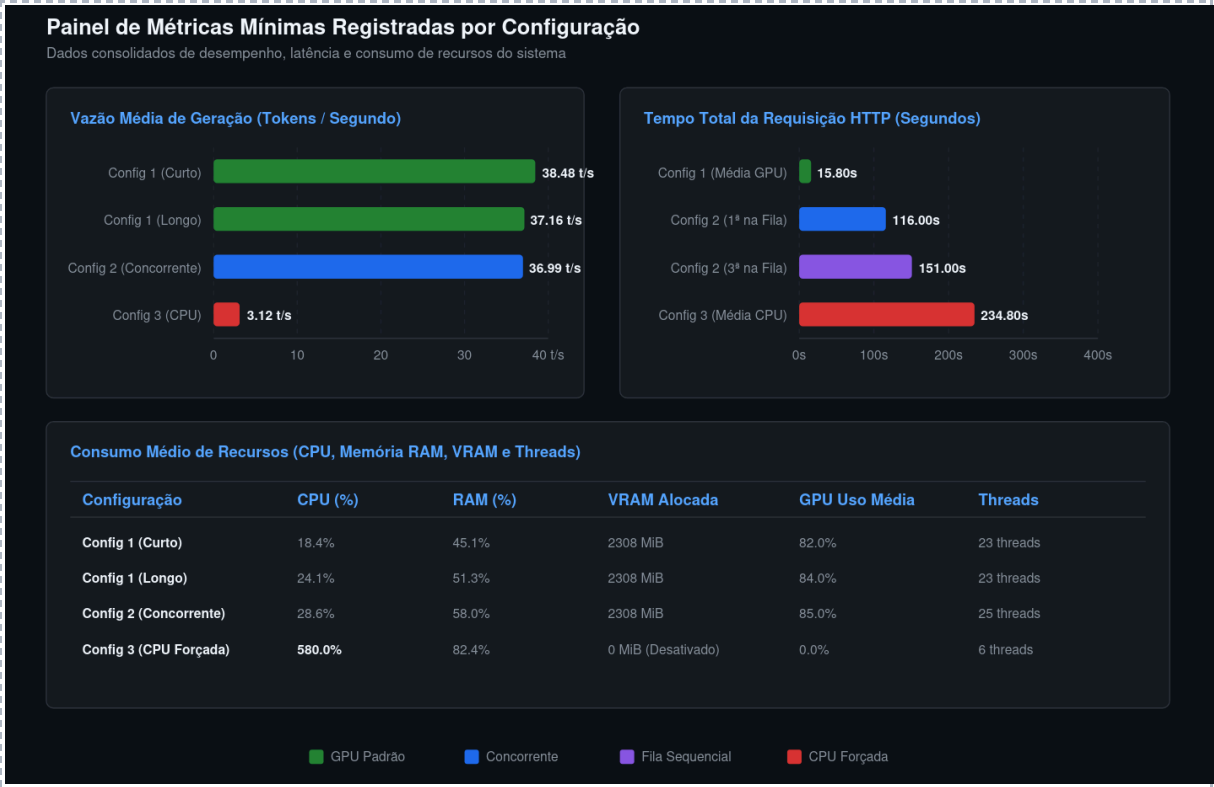
A avaliação da concorrência através da submissão simultânea de três requisições na Configuração 2 revelou que o aumento de carga simultânea não se traduz em ganho de vazão ou melhoria na responsividade global do sistema. Devido às restrições de memória de vídeo do hardware (4 GB de VRAM), o runtime operou sob a limitação de uma única execução paralela, acionando uma fila de processamento estritamente sequencial (FIFO). Embora a velocidade individual de geração de tokens se mantivesse estável em um patamar de 36 a 39 tokens por segundo em todas as requisições, o custo do tempo de espera em fila elevou severamente a latência de ponta a ponta percebida pelo usuário. A terceira requisição submetida aguardou cerca de 139 segundos na fila antes de ser iniciada, registrando um tempo total de resposta HTTP de 151 segundos, acompanhado de um leve aumento no consumo de memória RAM para 58% e de CPU para 28,6% a fim de sustentar o estado dos contextos mantidos em cache.

Sobre os sinais de competição por recursos do sistema, a alocação em GPU apresentou picos de uso entre 98% e 99% sem gerar instabilidade de memória, visto que a pegada do modelo Llama-3.2-3B quantizado ocupa aproximadamente 2,3 GB de VRAM. No entanto, o cenário de execução forçada em CPU gerou gargalos severos de desempenho: a ausência da GPU resultou na saturação contínua do processador principal, atingindo 580% de consumo de CPU (alocação máxima das 6 threads físicas ativas) e elevando o uso de memória RAM para 82,4%. Essa disputa pelos recursos computacionais da CPU causou uma degradação vertiginosa na taxa de geração, reduzindo a velocidade para um intervalo entre 2,04 e 5,27 tokens por segundo, além de estender o tempo de carregamento inicial (carga fria em memória RAM) para marcas de 124 a 185 segundos.

Ao longo dos nossos experimentos, ficou evidente o papel fundamental que a placa de vídeo (GPU) desempenha nesse cenário. Enquanto a RTX 3050 em agilidade, gerando respostas quase instantâneas a uma taxa fluida de aproximadamente 37 a 39 tokens por segundo, jogar essa mesma responsabilidade para o processador (CPU) fez o sistema "suar frio". Sem o acelerador dedicado, a

velocidade despencou para algo entre 2 e 5 tokens por segundo — um ritmo arrastado que torna a conversa lenta no dia a dia.

10.3 Gráfico



11. Discussão

Os resultados confirmam que a concorrência tem um custo alto no sistema. Na Configuração 2, as três requisições simultâneas não foram processadas em paralelo: o Ollama as colocou em uma fila, e uma delas sofreu falta de memória (swap de 782 MB), o que derrubou a velocidade de aproximadamente 27 tokens por segundo para 1 token/s e causou um erro de timeout em outra requisição. Isso mostra que, com pouca RAM disponível, a fila de espera tanto se torna mais lenta quanto pode travar o sistema.

Os dados de processos confirmam que quem faz o trabalho pesado é o llama-server, com 92,5% de CPU e 24 threads ativas. As muitas chamadas de sistema do tipo *futex* mostram as threads se sincronizando o tempo todo, e as chamadas de *clone* mostram a criação dessas threads. As chamadas de *mmap* indicam que o modelo é acessado direto do disco mapeado em memória, em vez de ser todo copiado pra RAM, o que explica por que a memória virtual (51,6 GB) é tão maior que a memória realmente usada (1 GB). As chamadas de rede (*accept4/poll*) refletem o streaming das respostas, palavra por palavra, entre o Ollama e o Open WebUI.

Quanto ao tamanho do prompt, os prompts longos demoraram bem mais no tempo total (até 25s) do que os curtos (2 a 4s), mas a velocidade em tokens/s ficou parecida entre os dois, ou seja, o prompt maior não deixa o modelo mais lento por token, só aumenta o tempo total porque gera mais tokens. Entre repetições, os prompts longos foram bem consistentes, mas o primeiro teste curto teve um

tempo total (12,26s) bem maior que o tempo de processamento interno (2,50s), sugerindo algum atraso externo (talvez o modelo "esquentando" na primeira chamada).

12. Relação com Sistemas Operacionais e IA generativa

Inicialmente, precisamos questionar o que os resultados trazidos até o momento tem a ver com Sistemas Operacionais. A execução da IA não ocorre de maneira isolada, tem vários recursos gerenciados pelo SO, como processador, arquivos, threads, comunicação por rede e chamadas de sistema, entre outros.

No ambiente estudado, a aplicação é composta pelos atores: Open WebUI é quem recebe a interação do usuário, tornando possível o recebimento das solicitações para que a API faça a transmissão, de modo que o Ollama faz a gerência dos chamados ao modelo de linguagem.

Um dos primeiros pontos de relação está no conceito de **processos**. Tanto o runtime do Ollama quanto os componentes utilizados pela aplicação são executados como processos, identificados pelo sistema operacional por meio de identificador de processo que acompanha o quantificador na verificação de processos ou mudanças durante a sua execução (PID). A análise da árvore de processos, juntamente com a identificação da observação de processo pai à medida que o comando `ps tree -p` é usado, a fim de observar a hierarquia dos processos (PPID) e identificar quais componentes são responsáveis por determinadas funções da aplicação. Essa observação é importante porque uma aplicação de IA generativa pode envolver diferentes processos para atender à interface, à comunicação com o runtime e à execução propriamente dita do modelo.

Além dos processos, a execução do modelo envolve o uso de **threads**, que representam unidades de execução pertencentes aos processos. A quantidade de threads observada em um processo permite analisar como as atividades da aplicação são distribuídas para execução pelo sistema operacional. Nesse contexto, a inferência de um modelo de linguagem pode envolver diversas tarefas que precisam ser executadas de maneira concorrente, fazendo com que o escalonador do sistema operacional determine quando e por quanto tempo cada thread terá acesso aos recursos de processamento.

Essa característica estabelece uma relação direta com os conceitos de **concorrência, paralelismo e escalonamento**. A concorrência se dá pela execução sequencial e disputada de um conjunto de tarefas independentes, através de um escalonador de processos, enquanto o paralelismo está relacionado à comunicação que mantém o código em vários processadores que executam tarefas ao mesmo tempo, destinado a fim de diminuir o tempo de série na execução das tarefas. Em uma aplicação de IA generativa, diferentes requisições podem chegar ao sistema enquanto o modelo está sendo carregado ou enquanto outra resposta está sendo produzida. Nessa situação, o sistema operacional precisa administrar a disputa pelos recursos disponíveis, o que pode refletir-se na latência, na vazão e na responsividade da aplicação. Por esse motivo, os experimentos de diferentes cargas e condições de concorrência permitem observar o comportamento da aplicação sob diferentes níveis de utilização dos recursos.

Outro conceito fundamental é a **gerência de memória**. Modelos de linguagem possuem arquivos e estruturas de dados que precisam ser carregados e utilizados durante a inferência. O tamanho do modelo, a quantidade de parâmetros, o nível de quantização e o tamanho do contexto de entrada podem influenciar a quantidade de memória necessária para sua execução. Assim, a análise do consumo de memória durante a inicialização, carregamento do modelo e geração de respostas permite relacionar diretamente o comportamento da IA com os mecanismos de gerenciamento de memória utilizados pelo sistema operacional.

A **quantização** também possui uma relação importante com a utilização dos recursos computacionais. Ao utilizar uma representação numérica mais compacta para os parâmetros do modelo, é possível reduzir o tamanho dos dados necessários para armazenamento e processamento. No experimento, a utilização de uma configuração quantizada permite analisar o compromisso entre os requisitos de recursos e o desempenho da inferência. Dessa maneira, uma característica específica do modelo de IA passa a produzir efeitos observáveis sobre recursos tradicionalmente estudados em Sistemas Operacionais, como memória, armazenamento e processamento.

A relação também pode ser observada na **entrada e saída de dados (E/S)**. Antes de realizar a inferência, o runtime precisa acessar arquivos relacionados ao modelo e a outros componentes necessários para sua execução. Durante o funcionamento da aplicação, também podem ocorrer operações relacionadas a configurações, diretórios e registros de execução. Essas atividades dependem das operações de entrada e saída disponibilizadas pelo sistema operacional e do gerenciamento do sistema de arquivos e do armazenamento.

As **chamadas de sistema** constituem outra ligação direta entre a aplicação de IA e o sistema operacional. Por meio delas, os programas solicitam ao kernel operações que não podem ser realizadas diretamente pelo código da aplicação. A utilização do **S** permite observar essas interações e identificar chamadas relacionadas à criação e execução de processos, acesso a arquivos, gerenciamento de memória e comunicação. Dessa forma, a análise das chamadas de sistema possibilita observar, em nível mais baixo, como o software de inferência utiliza os serviços fornecidos pelo sistema operacional.

A comunicação entre o Open WebUI e o Ollama também permite relacionar a aplicação aos conceitos de **comunicação em rede, portas e sockets**. O Open WebUI funciona como interface para o usuário e encaminha as solicitações ao runtime do Ollama por meio de uma API HTTP. O Ollama recebe essas requisições, gerencia o atendimento e utiliza o modelo carregado para produzir a resposta. Portanto, mesmo sendo uma aplicação executada localmente, existe uma comunicação entre componentes por meio de interfaces de rede, envolvendo portas, sockets e protocolos de comunicação.

A utilização conjunta de **Docker e WSL2** acrescenta outra camada à análise. A aplicação não está necessariamente concentrada em um único processo ou ambiente de execução. O Open WebUI é executado em um contêiner, enquanto o Ollama atua como runtime no ambiente WSL2. Essa arquitetura exige comunicação entre ambientes e demonstra como mecanismos de isolamento e virtualização podem fazer parte da execução de uma aplicação moderna de IA. A análise das portas, redes, volumes e diretórios persistentes permite observar como esses componentes são organizados pelo ambiente de execução.

Os **logs e os arquivos persistentes** também fazem parte dessa relação. Informações de configuração, dados da aplicação e registros de execução precisam ser armazenados em estruturas persistentes. Isso envolve o sistema de arquivos e operações de leitura e gravação, demonstrando que a execução de uma aplicação de IA generativa depende não apenas de processamento e memória, mas também de mecanismos de armazenamento e E/S.

Por fim, a análise de **múltiplas requisições ou conversas simultâneas** permite observar a relação entre IA generativa e gerenciamento de recursos sob concorrência. À medida que a quantidade de solicitações aumenta, os processos e threads envolvidos podem disputar CPU, memória, armazenamento e demais recursos disponíveis. O sistema operacional precisa administrar essa competição por meio de seus mecanismos de escalonamento e gerenciamento de recursos. As métricas de tempo de resposta, tempo até o primeiro token, tempo total e tokens por segundo possibilitam relacionar essa utilização dos recursos à responsividade e à vazão da aplicação.

Assim, a execução local de um modelo de linguagem pode ser compreendida como um caso prático de aplicação dos principais conceitos de Sistemas Operacionais. Processos e threads representam as unidades de execução; o escalonamento administra o acesso ao processamento; a gerência de memória disponibiliza os recursos necessários ao modelo; as operações de E/S permitem acessar modelos, configurações e registros; as chamadas de sistema fazem a ligação entre os programas e o kernel; e os mecanismos de comunicação possibilitam a interação entre o Open WebUI e o Ollama. Dessa forma, o estudo da IA generativa local permite observar como os conceitos tradicionalmente estudados em Sistemas Operacionais estão presentes em uma aplicação contemporânea e dependente de recursos computacionais.

13. Limitações e ameaças à validade

Dentre as dificuldades encontradas, sob carga de contexto elevado, o sistema apresentou swapping de memória (confirmado via `free -h`, 782MB em swap), causando degradação de ~25x na velocidade de geração (de ~27 para ~1 token/s) e um timeout HTTP 500 em outra requisição.

Em configuração com (CPU/RAM/VRAM ausentes):

A ausência do monitoramento de CPU/RAM/VRAM durante o teste de concorrência impede uma análise completa do comportamento dos recursos do sistema nesta configuração. Como o `monitor.sh` só foi iniciado às 19:29, enquanto as três requisições simultâneas ocorreram entre 18:51 e 18:54, não é possível afirmar, a partir dos dados coletados, quanto de CPU e RAM foi utilizado durante a concorrência.

Entretanto, os dados de tempo de resposta e tokens/s continuam válidos, pois foram obtidos por outra fonte. Assim, é possível avaliar o desempenho da inferência, mas não relacioná-lo diretamente ao consumo de recursos do sistema. Portanto, a principal consequência é a impossibilidade de determinar se a degradação observada ocorreu por saturação de CPU, pressão de memória, concorrência entre processos ou combinação

Em configuração com (CPU/RAM não confiáveis; GPU/VRAM válidos):

Nesta configuração, o problema está relacionado à forma como o Ollama organiza seus processos. O monitoramento acompanhou principalmente o processo `ollama serve`, enquanto a inferência pesada foi executada pelo processo filho `llama-server`. Dessa forma, os valores próximos de 0% de CPU e RAM registrados pelo monitor não representam o consumo real da inferência.

Isso limita a análise de CPU e RAM, pois não é possível utilizar esses valores para concluir que o modelo consumiu poucos recursos. Por outro lado, os dados de GPU e VRAM permanecem relevantes: como a GPU apresentou 0% de utilização e a VRAM permaneceu sem uso, eles sustentam a conclusão de que a inferência ocorreu em CPU-only, conforme configurado com `CUDA_VISIBLE_DEVICES=-1`.

13.1 Impacto geral

As duas limitações afetam principalmente a correlação entre desempenho e utilização de recursos. Ou seja, os resultados permitem observar que houve diferenças de velocidade, tempo de resposta, concorrência e ocorrência de swapping, mas não permitem, em todas as configurações, estabelecer com precisão qual recurso do sistema foi responsável por cada variação de desempenho.

Ainda assim, o resultado do swapping é particularmente importante. A identificação de aproximadamente 782 MB de swap durante carga elevada fornece uma evidência direta de pressão de memória. A queda de aproximadamente 27 para 1 token/s, cerca de 25 vezes, juntamente com o timeout HTTP 500, demonstra que a pressão de memória teve impacto significativo na estabilidade e no desempenho da inferência.

Em termos de Sistemas Operacionais, as limitações mostram a importância de monitorar corretamente processos e processos filhos, gerenciamento de memória virtual, swap e utilização de recursos. O caso também evidencia que monitorar apenas o processo pai de uma aplicação não é suficiente quando o trabalho computacional é delegado a processos filhos.

14. Declaração de uso de IA generativa

Ferramenta e modelo utilizados	Claude code(Sonnet), Gemini 3.6 Flash/ 3.1 Pro e ChatGPT GPT-5.6 Luna.
Finalidade geral do uso	Instalação, busca de referências, tradução, idealização de estrutura textual.

14.1 Prompts relevantes

Nº	Prompt (resumido)	Finalidade	Síntese da sugestão obtida	Uso (aceito / corrigido / rejeitado)
1	Diagnóstico do erro host.docker.internal	Depuração/troubleshooting	Sequência de comandos (ss, curl -v, hostname -I) que revelou Docker Desktop e WSL2 em redes distintas	aceito
2	Traduzir documentação oficial	Tradução	Textos traduzidos	aceito
3	Como forçar o Ollama a rodar somente em CPU, sem usar a GPU, para a Configuração 3 do experimento	Configurar a Configuração 3 (execução forçada em CPU) do experimento comparativo	Primeira sugestão apontou o parâmetro num_gpu=0, encontrado em discussões de comunidade (não documentação oficial). Após verificação em docs.ollama.com/gpu, a resposta foi corrigida para a variável de ambiente oficial CUDA_VISIBLE_DEVICES=-1, usada para lançar a instância	Corrigido pois a sugestão inicial não-oficial foi substituída pela solução documentada oficialmente antes de ser aplicada
4	Relacionar o uso de memória e CPU durante a inferência do Ollama com os conceitos de Sistemas Operacionais.	Alinhar os resultados experimentais aos conteúdos de SO e IA generativa.	A IA mostrou o consumo de CPU e memória aos conceitos de SO, porém apresentou uma explicação predominantemente teórica e genérica, sem considerar suficientemente os resultados específicos obtidos nos testes.	Corrigido por insuficiência
5	Pedido de rascunho inicial das seções 11 e 13 com base nos dados de processos, chamadas	Obter um ponto de partida estruturado	Texto conectando os números já coletados pela equipe (fila FIFO, swap, chamadas de sistema, quantização) aos pontos exigidos no roteiro do enunciado	Aceito e utilizado

Nº	Prompt (resumido)	Finalidade	Síntese da sugestão obtida	Uso (aceito / corrigido / rejeitado)
	de sistema e resultados coletados	para as duas seções		

14.2 Verificação

Sugestões corrigidas ou rejeitadas	Foi cogitado usar o parâmetro <code>num_gpu</code> (não-oficial, de fóruns da comunidade) para forçar execução em CPU; a sugestão foi rejeitada após verificação na documentação oficial do Ollama, adotando <code>CUDA_VISIBLE_DEVICES=-1</code> ,
Erros encontrados nas respostas da IA	A sugestão inicial (<code>num_gpu</code>) veio de fontes não-oficiais (fóruns/comunidade) e não foi sinalizada de imediato como não-verificada; só após questionamento da confiabilidade é que a resposta foi checada contra a documentação oficial do Ollama
Testes, documentação ou observações usados para verificação	Consulta direta à documentação oficial (docs.ollama.com/gpu), confirmando <code>CUDA_VISIBLE_DEVICES=-1</code> como mecanismo suportado; validado na prática rodando a instância e conferindo em <code>ollama ps</code> que o uso de GPU ficou em 0%, além de busca por fontes seguras como a bibliografia disposta no final do documento.

14.3 Distribuição das contribuições dos integrantes no uso de IA

Carlos Henrico Fontes Cabral	Utilizou o Gemini para auxiliar no diagnóstico dos parâmetros do ambiente Ollama/Open WebUI
Daniel de Andrade Lima	Utilizou a Inteligência Artificial (Google Gemini) para auxiliar no processamento de tabelas e dados, ideação e estruturação de texto, pesquisas básicas de terminologia e tradução textual.
Lauren Victoria Ferreira Roma	Utilizou IA generativa (Claude Code) para pesquisar e verificar o método de execução forçada em CPU usado na Configuração 3 do experimento e para diagnóstico e configuração do ambiente WSL2/Docker/Ollama.
Marcos Vinícios Assunção Nery Santos	Utilizou a Inteligência Artificial (Google Gemini) como suporte técnico para o diagnóstico de erros de rede, liberação de portas e configuração inicial da integração entre WSL2, Docker e Ollama. Adicionalmente, empregou a ferramenta como apoio para o brainstorming de tópicos, revisão e organização lógica das ideias que fundamentaram a escrita das Seções 2 e 15.
Roseane Resende Couto	Utilizou a Inteligência Artificial (Claude, Anthropic) para auxiliar no refinamento da discussão dos resultados, na estruturação textual dos dados coletados e na simplificação de linguagem técnica da Seção 11.

Wanessa Silva Santos

Utilizou a IA(ChatGPT GPT-5.6 Luna) como apoio à análise e organização das informações técnicas, à relação entre os conceitos de Sistemas Operacionais e IA generativa, mantendo a conferência dos resultados com as evidências obtidas pela equipe.

15. Conclusão

A execução deste experimento permitiu responder de forma abrangente à pergunta norteadora do projeto, evidenciando que a responsividade de um sistema local de IA generativa é estritamente ditada pela interação entre as configurações da aplicação, as características do modelo e o gerenciamento de recursos pelo sistema operacional. Ficou comprovado que a inferência ocorre através de uma delegação de responsabilidades estruturada pelo SO, onde o processo daemon ollama atua como escuta e gerenciador, e o processo filho llama-server assume a carga computacional real. O escalonador do SO ajustou ativamente essa demanda instanciando threads conforme a restrição de hardware: mantendo 24 threads em condições de aceleração gráfica e apenas 6 threads físicas ativas via OpenMP durante a restrição em CPU.

O formato do modelo (GGUF com quantização Q4_K_M) aliado a chamadas de sistema eficientes (como o mmap) foram essenciais para viabilizar o sistema na infraestrutura limitada da equipe (4 GB de VRAM e 3,4 GB de RAM visíveis no WSL2), pois permitiram mapear o arquivo do disco direto para a memória, otimizando o armazenamento e o acesso aos parâmetros. Sob aceleração de hardware (RTX 3050), o sistema manteve excelente responsividade, alcançando vazões de 36 a 39 tokens por segundo. Em contraste, a configuração que forçou a execução exclusiva na CPU saturou severamente os recursos, elevando o uso do processador a 580% e derrubando drasticamente a geração para a faixa de 2 a 5 tokens por segundo.

A avaliação de múltiplas requisições (concorrência) revelou o custo crítico da sobrecarga de contexto. Sem memória de vídeo suficiente para processamento paralelo, o sistema foi forçado a adotar uma fila sequencial (FIFO), onde as requisições aguardaram mais de 100 segundos até serem atendidas. A pressão na memória física obrigou o SO a utilizar paginação (com 782 MB de swap registrados), degradando a vazão para apenas 1 token por segundo e culminando em timeouts (HTTP 500). Esses dados confirmam que a concorrência em IA generativa local esbarra rapidamente nas limitações de paralelismo do hardware, gerando gargalos de latência.

Como possíveis extensões deste trabalho, recomenda-se investigar experimentalmente modelos de linguagem menores (como as arquiteturas da família de 1B de parâmetros) ou níveis de quantização ainda mais compactos (como Q2 ou Q3) para observar se a mitigação do consumo de RAM/VRAM elimina a dependência do swap durante cargas altamente concorrentes. Ademais, a replicação das metodologias aqui descritas em um sistema operacional Linux instalado nativamente — livre da camada de virtualização do WSL2 — proporcionaria um controle mais refinado e maior confiabilidade sobre a captura das métricas fundamentais de uso de RAM e CPU sob estresse.

16. Referências

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. Operating System Concepts. 10. ed. Hoboken: Wiley, 2018.

TANENBAUM, Andrew S.; BOS, Herbert. Modern Operating Systems. 5. ed. Boston: Pearson, 2023.

KERRISK, Michael. The Linux Programming Interface. San Francisco: No Starch Press, 2010.

LINUX MAN-PAGES PROJECT. fork(2), execve(2), wait(2), clone(2), pthreads(7), sched(7). Disponível em: <https://man7.org/linux/man-pages/>.

THE LINUX KERNEL DOCUMENTATION. Scheduler documentation. Disponível em: <https://docs.kernel.org/scheduler/>.

OLLAMA. Repositório oficial. Disponível em: <https://github.com/ollama/ollama>.

OLLAMA. Documentação da API. Disponível em: <https://docs.ollama.com/api/introduction>.

OPEN WEBUI. Repositório oficial. Disponível em: <https://github.com/open-webui/open-webui>.

OPEN WEBUI. Documentação oficial. Disponível em: <https://docs.openwebui.com/>.

HUGGING FACE. Models. Disponível em: <https://huggingface.co/models>.

NIST. Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile. NIST AI 600-1, 2024. Disponível em: <https://doi.org/10.6028/NIST.AI.600-1>.

IBM. O que é computação paralela? IBM Think, 2025. Disponível em: <https://www.ibm.com/br-pt/think/topics/parallel-computing>. Acesso em: 9 set. 2026.

SHOTTS, William. The Linux Command Line. LinuxCommand.org. Disponível em: <https://linuxcommand.org/tlcl.php>.